

Machine Learning Operations (MLOps) AIMLCZG523

Assignment 01

Total Marks 50

MLOps Experimental Learning Assignment: End-to-End ML Model Development, CI/CD, and Production Deployment Experimental Learning.

Objective: Design, develop, and deploy a scalable, reproducible machine learning solution using modern MLOps best practices. The assignment emphasizes practical automation, experiment tracking, CI/CD pipelines, containerization, cloud deployment, and monitoring, mirroring real-world production scenarios.

Dataset: Title: Heart Disease UCI Dataset

- Source: [UCI Machine Learning Repository](#)
- CSV containing 14+ features (age, sex, blood pressure, cholesterol, etc.) and a binary target (presence/absence of heart disease).

Problem Statement: Build a machine learning classifier to predict the risk of heart disease based on patient health data, and deploy the solution as a cloud-ready, monitored API.

Assignment Tasks

1. Data Acquisition & Exploratory Data Analysis (EDA) [5 marks]

- Obtain the dataset (provide download script or instructions). Clean and preprocess the data (handle missing values, encode features). Perform EDA with professional visualizations (histograms, correlation heatmaps, class balance).

2. Feature Engineering & Model Development [8 marks]

- Prepare final ML features (scaling, encoding). Build and train at least two classification models (e.g., Logistic Regression and Random Forest). Document model selection and tuning process. Evaluate using cross-validation and relevant metrics (accuracy, precision, recall, ROC-AUC).

3. Experiment Tracking [5 marks]

- Integrate MLflow (or similar) for experiment tracking. Log parameters, metrics, artifacts, and plots for all runs.

4. Model Packaging & Reproducibility [7 marks]

- Save the final model in a reusable format (e.g., MLflow, pickle, ONNX). Write a clean requirements.txt (or Conda env file). Provide preprocessing pipeline/transformers to ensure full reproducibility.



5. CI/CD Pipeline & Automated Testing [8 marks]

- Write unit tests for data processing and model code (Pytest or unit test). Create a GitHub Actions (or Jenkins) pipeline: Linting, unit test, and model training steps. Artifacts/logging for each workflow run.

6. Model Containerization [5 marks]

- Build a Docker container for the model serving API (Flask or FastAPI recommended). Expose /predict endpoint, accept JSON input, return prediction and confidence. Container must build and run locally with sample input.

7. Production Deployment [7 marks]

- Deploy the Dockerized API to a public cloud or local Kubernetes (GKE, EKS, AKS, or Minikube/Docker Desktop). Use a deployment manifest or Helm chart. Expose via Load Balancer or Ingress. Verify endpoints and provide deployment screenshots.

8. Monitoring & Logging [3 marks]

- Integrate logging of API requests. Demonstrate simple monitoring (Prometheus + Grafana or API metrics/logs dashboard).

9. Documentation & Reporting [2 marks]

- Submit a professional Markdown or PDF report including:
 - Setup/install instructions.
 - EDA and modelling choices.
 - Experiment tracking summary.
 - Architecture diagram.
 - CI/CD and deployment workflow screenshots.
 - Link to code repository.

Deliverables

- GitHub repository with:
 - Code, Dockerfile(s), requirements.txt/env.yml
 - Cleaned dataset and download script/instructions
 - Jupyter notebooks/scripts (EDA, training, inference)
 - test/ folder with unit tests
 - GitHub Actions workflow YAML (or Jenkinsfile)
 - Deployment manifests/Helm charts
 - Screenshot folder for reporting
 - Final written report 10 pages as a doc/docx/pdf file.
- Deployed API URL (if public) or access instructions (for local testing)
- Include a short video recording of the overall pipeline

Production-Readiness Requirements

- All scripts must execute from clean setup using requirements file.
- Model must serve correctly in an isolated environment (Docker container build/test proof required).
- Pipeline must fail on code or test errors and give clear logs.

Academic Integrity Note:

- Students are expected to complete this assignment independently and demonstrate their own understanding of MLOps concepts and implementation practices. While discussions for learning purposes are encouraged, direct copying of code, notebooks, visualizations, feature engineering steps, CI/CD workflows, deployment configurations, or overall project pipelines from other students or online sources is strictly prohibited.
- Assignments exhibiting highly similar implementations including identical feature selections, EDA visualizations, model pipelines, Docker/Kubernetes configurations, API structures, reports, or GitHub workflows may be treated as copied submissions and will be subject to academic integrity review and grading penalties.
- Students are encouraged to personalize their approach, experimentation process, architecture decisions, monitoring setup, and documentation to reflect originality and individual effort.



Machine Learning Operations (MLOps) AIMLCZG523

Assignment 01

FAQs

Q. Which dataset should be used?

The assignment requires you to use the **Heart Disease UCI Dataset** from the UCI Machine Learning Repository.

Q. What kind of Exploratory Data Analysis (EDA) is expected?

You should include:

- Histograms
- Correlation heatmaps
- Class distribution plots
- Missing value analysis
- Feature relationship analysis

Professional visualizations using Matplotlib, Seaborn, or Plotly are recommended.

Q. Should we clean and preprocess the data?

Yes. You should:

- Handle missing values
- Encode categorical variables
- Scale numerical features (if applicable)
- Split train/test data properly

Q. How many models should be trained?

At least two classification models are expected.

Examples:

- Logistic Regression
- Random Forest
- XGBoost
- Support Vector Machine or any choice of yours

Q. Which evaluation metrics should be used?

Recommended metrics:

- Accuracy
- Precision
- Recall
- F1-score
- ROC-AUC

Cross-validation is encouraged.

Q. Do we need hyperparameter tuning?

Yes. Basic tuning using:

- GridSearchCV
- RandomizedSearchCV
- Optuna (optional) or any other

is recommended.

Q. Why are we using MLflow?

MLflow helps track:

- Experiments
- Parameters
- Metrics
- Model versions
- Artifacts

This ensures reproducibility and proper experiment management.

Q. What should be logged in MLflow?

You should log:

- Model parameters
- Accuracy metrics
- ROC curves/plots
- Confusion matrix
- Saved models

Q. Why do we need requirements.txt?

It allows the project to run in a clean environment with all required dependencies installed consistently.

Q. Should preprocessing steps also be saved?

Yes. Preprocessing pipelines must be reusable during inference.

Recommended approach:

- sklearn Pipeline
- ColumnTransformer

Q. Which model serialization format can be used?

Any of the following are acceptable:

- Pickle
- Joblib
- MLflow model format
- ONNX

Q. Which framework should be used for the API?

Recommended:

- FastAPI (preferred)
- Flask

Q. What should the API expose?

A /predict endpoint that:

- Accepts JSON input
- Returns prediction
- Returns confidence/probability score

Q. How can the API be tested?

You may use:

- Postman
- curl
- Swagger UI (FastAPI)
- pytest

Q. Why are we using Docker?

Docker ensures the application runs consistently across environments by packaging:

- Code
- Dependencies
- Runtime environment

inside a container.

Q. What should be included in the Docker container?

The container should include:

- API code
- Trained model
- Dependencies
- Inference pipeline

Q. How do we verify Docker works? You should:

1. Build the Docker image
2. Run the container locally
3. Test the /predict endpoint successfully

Q. What is expected in the CI/CD pipeline?

The pipeline should automate:

- Code linting
- Unit testing
- Model training (optional)
- Build validation

Q. Which CI/CD tools are allowed?

You may use:

- GitHub Actions (recommended)
- Jenkins
- GitLab CI/CD

Q. Should the pipeline fail if tests fail?

Yes. The pipeline must stop and clearly report errors if:

- Tests fail
- Code quality checks fail

Q. Is cloud deployment mandatory?

You may deploy using:

- Public cloud (AWS/GCP/Azure)
- OR
- Local Kubernetes (Minikube/Docker Desktop Kubernetes)

Q. Which Kubernetes platforms are acceptable? Examples:

- Minikube
- Docker Desktop Kubernetes
- Amazon EKS
- Google GKE
- Azure AKS

Q. What Kubernetes files are expected?

You should provide:

- Deployment YAML
- Service YAML
- Optional Helm chart

Q. What monitoring is expected?

Basic monitoring such as:

- API request logging
- Response monitoring
- Prometheus metrics
- Grafana dashboards

is sufficient.

Q. Why is monitoring important in ML systems?

Monitoring helps detect:

- Model failures
- Data drift
- API downtime
- Performance degradation

Q. What should the final report contain?

The report should include:

- Project overview
- EDA findings
- Model comparison
- Architecture diagram
- MLflow screenshots
- CI/CD screenshots
- Deployment screenshots
- Setup instructions
- Repository link



Q. Is a GitHub repository mandatory?

Yes. A clean and properly organized repository is expected.

Recommended Tool Stack

Category	Recommended Tools
Programming Language	Python
Notebook Environment	Jupyter Notebook, VS Code
Data Processing	Pandas, NumPy
Visualization	Matplotlib, Seaborn, Plotly
ML Frameworks	Scikit-learn, XGBoost
Experiment Tracking	MLflow
API Framework	FastAPI, Flask
Testing	Pytest
Containerization	Docker
CI/CD	GitHub Actions, Jenkins
Orchestration	Kubernetes, Minikube
Monitoring	Prometheus, Grafana
Version Control	Git, GitHub
Deployment Platforms	AWS, GCP, Azure, Render, Railway